

Programación Avanzada
IIC2233 2026-2

Anuncios

27 de agosto de 2026



- Hoy tendremos la primera **¡Actividad evaluada!**
 - Suban la actividad a la carpeta **Actividades/AC1/**
 - La carpeta AC1 solo debe tener la actividad de hoy.
 - Recuerden **marcar su salida con la TUC**, sino quedarán como ausentes.
-

Control de Entrada

Solo lápiz y papel. No pueden utilizar apuntes, computadores, ni cualquier otro dispositivo electrónico.

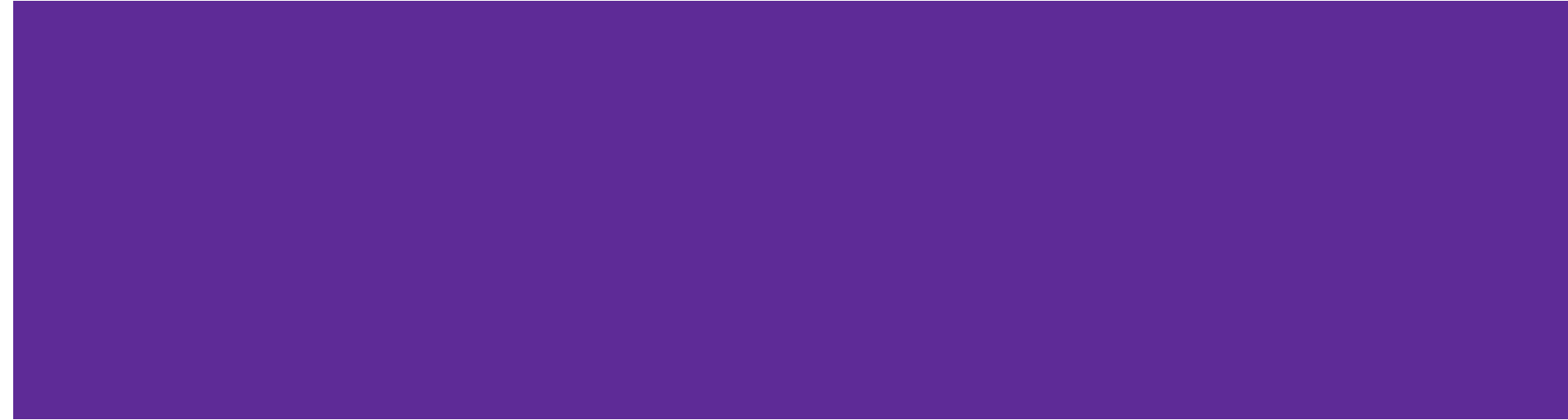


Comentarios AC1

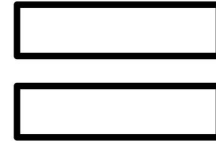
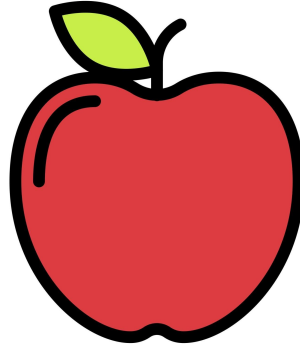
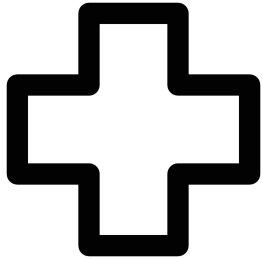
- Suban la actividad a la carpeta **Actividades/AC1/**
La carpeta AC1 solo debe tener los archivos de esta actividad.
Si la actividad **AC1Diagnostico** está ahí, debes eliminar esos archivos.
- Recuerden que esto es una **actividad individual.** **Mantengan el silencio.**
- Puedes copiar una carpeta desde Syllabus a tu repositorio usando el comando en consola `cp -r <ruta carpeta_origen> <ruta carpeta_destino>`

Cierre

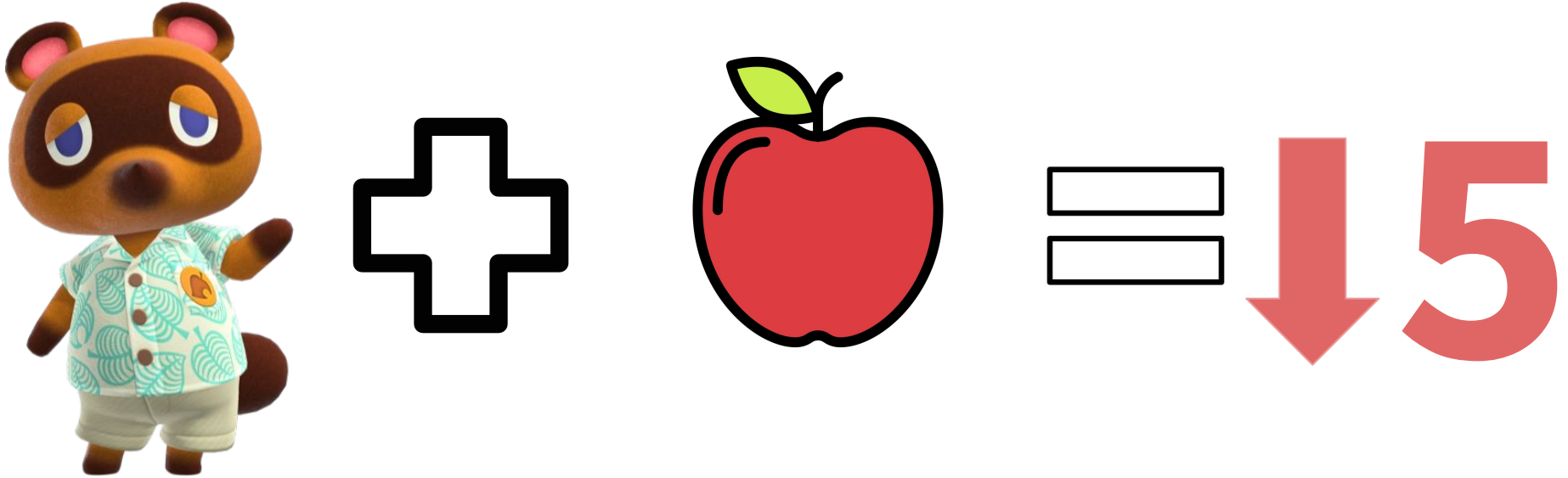
Actividad evaluada 1: OOP Avanzados.



Que debemos modelar?



Que debemos modelar?



Parte 1: Regalo



```
1 class Regalo:
2
3     def __init__(self, nombre: str, categoria: str, precio: int) → None:
4         """
5         POR COMPLETAR (Parte 1.1):
6         """
7         pass
8
9     # POR COMPLETAR (Parte 1.2):
10
11     # POR COMPLETAR (Parte 1.3):
12
```

Parte 1: Regalo



```
1 class Regalo:
2
3     def __init__(self, nombre: str, categoria: str, precio: int) → None:
4
5         # POR COMPLETAR (Parte 1.1):
6         self.nombre = nombre
7         self.categoria = categoria
8         self._precio = 0
9         self.precio = precio
10
```


Parte 1: Regalo



```
1 # POR COMPLETAR (Parte 1.2):
2     @property
3     def precio(self) → int:
4         return self._precio
5
6     @precio.setter
7     def precio(self, nuevo_precio: int) → None:
8         if 0 ≤ nuevo_precio ≤ PRECIO_MAXIMO:
9             self._precio = nuevo_precio
10        else:
11            print(
12                f"[Aviso] {nuevo_precio} no es un precio valido '
13                f"(0 a {PRECIO_MAXIMO}). "
14                f"Se mantiene {self._precio}."
15
```

Parte 1: Regalo



```
1
2     # POR COMPLETAR (Parte 1.3):
3     def __str__(self) → str:
4         return f"{self.nombre} ({self.precio})"
5
6     def __repr__(self) → str:
7         return (
8             f"Regalo(nombre='{self.nombre}', "
9             f"categoria='{self.categoria}', "
10            f"precio={self.precio})"
11        )
12
```

Parte 2: Aldeanos



```
1 class Rosie(Alegre):
2     """
3     Aldeana alegre.
4     """
5
6     def __init__(self) → None:
7         # POR COMPLETAR (Parte 2)
8         pass
9
10
```

Parte 2: Aldeanos



```
1 class Rosie(Alegre):
2     """
3     Aldeana alegre.
4     """
5
6     def __init__(self) → None:
7         super().__init__(nombre="Rosie", especie="gato")
8
```

Parte 3: Personalidades

```
1 class Cascarrabias(Aldeano):
2     """
3     Aldeanos a los que solo los impresionan los regalos caros
4     """
5
6     def __add__(self, regalo: Regalo) → Interaccion:
7         if regalo.precio ≥ PRECIO_CARO:
8             puntos = 10
9         else:
10             puntos = -5
11
12         self.sumar_amistad(puntos)
13         return Interaccion(self, regalo, puntos)
14
```

Parte 3: Personalidades



```
1 class Alegre(Aldeano):
2
3     def __add__(self, regalo: Regalo) → Interaccion:
4         if regalo.categoria == "fruta":
5             puntos = 12
6         else:
7             puntos = 8
8
9         self.sumar_amistad(puntos)
10        return Interaccion(self, regalo, puntos)
11
12
```

Parte 3: Personalidades



```
1 class Presumida(Aldeano):
2
3     def __add__(self, regalo: Regalo) → Interaccion:
4         if regalo.categoria ≠ "ropa":
5             puntos = 1
6         elif regalo.precio ≥ PRECIO_CARO:
7             puntos = 15
8         else:
9             puntos = 6
10
11     self.sumar_amistad(puntos)
12     return Interaccion(self, regalo, puntos)
13
14
```